SA4110 MACHINE LEARNING APPLICATION DEVELOPMENT

CONTINUAL ASSIGNMENT – GROUP 3 PROJECT – IMAGE CLASSIFIER

Completed by: SA62 Group 3

Khairulanwar Bin Kamaruzaman, Amelia Wong, Soo Kwok Heng, Chai Lee Yeo, Huayuan Xie, Maowei Yang

Table of Contents

Declaration of Use of Generative AI

The group acknowledges the use of Generative AI (Gemini) to augment and debug errors, and to assist in fruit classification continual assignment, which includes code optimization and debugging. AI was utilized strictly as a supplementary tool to support the development process. All core concepts, final decisions, and underlying logic within this project represent the original work of the group members, who take full responsibility for the accuracy and integrity of the final output.

Project Set Up & Image Preprocessing

Environment and Imports

1. Verify and install missing packages required to run the code; Import required libraries.
2. Configured Keras to use PyTorch backend, and session clearing as well as random seeding.

Image Preprocessing

1. **Visual Inspection:** Training and testing images are visually inspected to ensure that the listed file name matches the image precisely.

2. **Data Quality Handling:** Upon inspection, we found that two images, banana_35.jpg and banana_61.jpg, contain multiple fruit classes each, and banana_35.jpg contains a fruit class that is out of scope: "Peach". As the rest of the images are single-class, we decided to **exclude** these images from preprocessing to maintain dataset consistency.

banana_35.jpg	banana_61.jpg
	

3. **Data Preparation:** The remaining 218 training images are split into Training (80%) and Validation (20%) Datasets. The class distribution was generally balanced, so no resampling was required.

Class Label	Training Images	Validation Images	Testing Images
Apple	60	15	19
Banana	57	14	18
Orange	57	15	18
Total	**174**	**44**	**55**

The FruitLabeller class maps class strings to integer labels and returns tensors, while DataLoaders are built with seeded generators for better stability.

4. **Image Resizing & Normalisation:** All images undergo historically accepted practice of image transformation of (224 × 224 pixels). Concurrently, this ensures that images are architecturally compatible with existing pre-trained models. Normalisation mean/std are computed from the training set and applied consistently for custom models.

Experiments

1. Baseline Model

Model Overview: This model aimed to achieve a strong foundational architecture to increase its accuracy and isolate the effects of further experimentation.

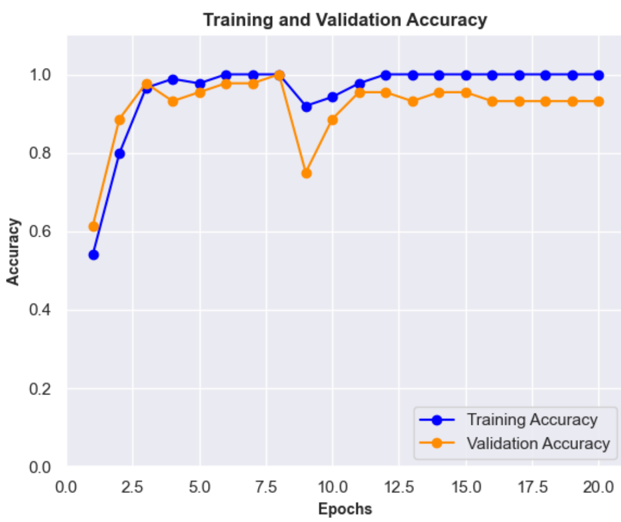
Findings:

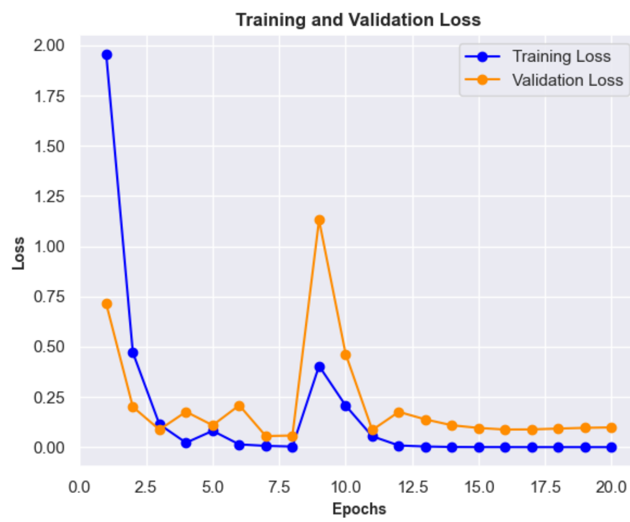
- Applying a consistent preprocessing pipeline with normalized transforms stabilized distribution of input and improved convergence speed.
- With three conv blocks and a dense head, the baseline has sufficient capacity for the three fruit classes.
- Batch size had a measurable effect on optimization. Among the values explored, a batch size of 16 offered the best speed-accuracy trade-off, achieving efficient training while preserving model accuracy.

Architecture: The baseline model utilizes a 3-block Convolutional Neural Network (CNN) architecture. Each blockconsists of a **Conv2D** layer (32, 64, and 128 filters respectively) paired with **MaxPooling2D** for down-sampling. The network concludes with a **flattened dense** layer of 128 nodes and a 3-node **SoftMax** output.

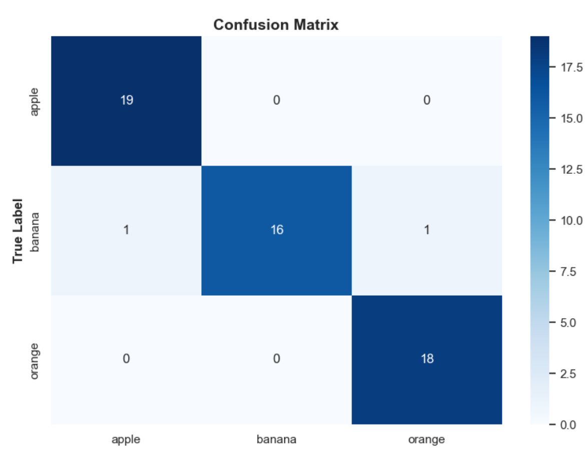
- Learning Rate: 0.001 + Dropout Rate: 0.0 (Non-regularised)

Results:





Class	Precision	Recall	F1-Score	Support
Apple	0.9500	1.0000	0.9740	19
Banana	1.0000	0.8890	0.9410	18
Orange	0.9470	1.0000	0.9730	18



Final Test Loss: 0.2307

Final Test Accuracy: 0.9636

Analysis:

a. **Overall Performance:** The baseline model demonstrates strong predictive capability on unseen data, achieving a high final test accuracy of 0.9636 (96.36%) and a relatively low final test loss of 0.2307. This indicates that the core 3-block CNN architecture is structurally sound for extracting the necessary features for this specific fruit classification task, though its late-stage efficiency degrades without regularization.
b. **Mid-Training Instability:** The training process experienced severe disruption at Epoch 9. Validation loss spiked significantly, exceeding 1.0, and validation accuracy dropped to roughly 0.75. Training loss also saw a corresponding spike. While the network successfully recovered by Epoch 12, this volatility suggests that the model is highly sensitive and may occasionally overshoot optimal minimums.
c. **Signs of Overfitting:** Signs of overfitting are visible in the latter half of the training run. From Epoch 13 onwards, training accuracy locks in at a perfect 1.0 (100%), and training loss drops flat to absolute zero. Meanwhile, validation accuracy plateaus beneath the training line and validation loss ceases to improve. This indicates that the model spent the last 7-8 epochs memorizing the training images rather than learning generalizable features.

2. Improved Model with Hyperparameter Tuning

Model-Level Overview: This experiment aimed to improve the baseline CNN through hyperparameter tuning while keeping the original CNN architecture unchanged. Learning rate and dropout settings were compared to assess the effects of optimization and regularization.

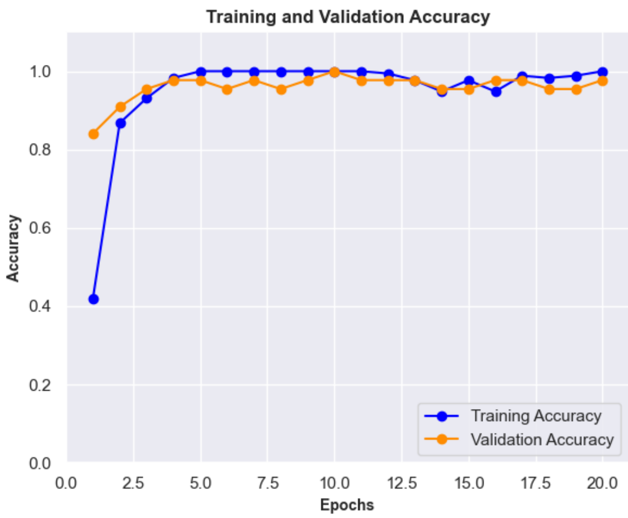
Findings:

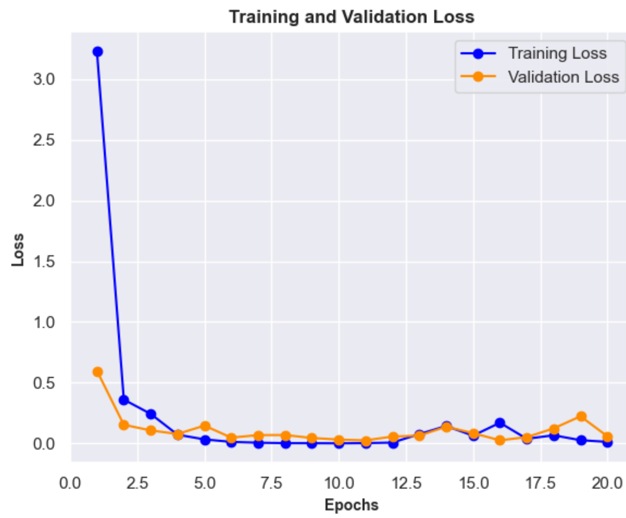
- Learning rate tuning had a clear effect on stability. The Learning Rate 0.01 Model performed poorly, with 32.73% accuracy and 10.8431 loss, while the Learning Rate 0.0001 Model improved to 92.73% accuracy but still did not outperform the baseline. This suggests that an unsuitable learning rate could severely weaken learning.
- Dropout results showed that moderate regularisation was preferable. Dropout 0.3 and 0.5 both achieved 96.36% accuracy, but Dropout 0.3 had lower loss than Dropout 0.5. Dropout 0.7 reduced accuracy to 94.55%, suggesting that excessive dropout weakened feature learning.

Architecture: The selected improved model utilizes the same 3-block CNN architecture established in the baseline model. The core modification for this experiment is the implementation of a Dropout layer configured with a rate of 0.3, which randomly deactivate 30% of the neurons during training, immediately preceding the final 3-node SoftMax output layer.

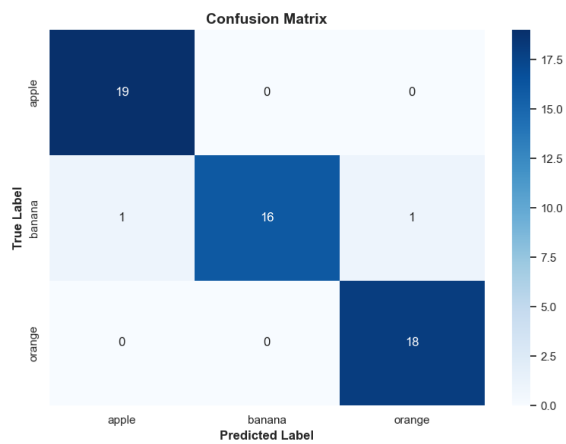
- Learning Rate: 0.001 **+** Dropout Rate: 0.3

Results:





Class	Precision	Recall	F1-Score	Support
Apple	0.9500	1.0000	0.9740	19
Banana	1.0000	0.8890	0.9410	18
Orange	0.9470	1.0000	0.9730	18



Final Test Loss: 0.2225
Final Test Accuracy: 0.9636

Analysis:

a. **Overall Performance:** The application of a 0.3 dropout rate yielded strong results, demonstrating robust generalization to unseen data. The model reached a high final test accuracy of of 0.9636 (96.36%) and recorded a final test loss of 0.2225. The overall result indicates that the 0.3 dropout rate preserved peak classification performance while actively forcing the network to learn redundant representations.

b. **Mid-Training Instability:** The training process was highly stable for the first half of the run. Minor fluctuations in validation loss and accuracy appeared later in the training cycle, most notably around epochs 14 and 19. However, these spikes were minor and the model recovered immediately, indicating that the architecture handles the noise introduced by the dropout layer predictably.

c. **Signs of Overfitting:** There are no signs of late-stage overfitting. The 0.3 dropout rate successfully acted as a strong regularizer. Unlike the unregularized baseline which began to memorize the training data, the training and validation loss curves here remained tightly coupled all the way through epoch 20. The divergence typical of overfitting was successfully prevented.

3. Data Augmentation Model

Model-Level Overview: This experiment aimed to evaluate whether data augmentation could improve the robustness and generalization of the regularized CNN while keeping its original architecture. It examined image-level variation including geometric, photometric, and Gaussian blur modifications. Geometric augmentation evaluates robustness to spatial changes and blur, while photometric augmentation evaluates robustness to brightness and contrast changes. Geometric augmentation without Gaussian blur was selected as the representative because it maintained strong test accuracy while introducing explicit regularization.

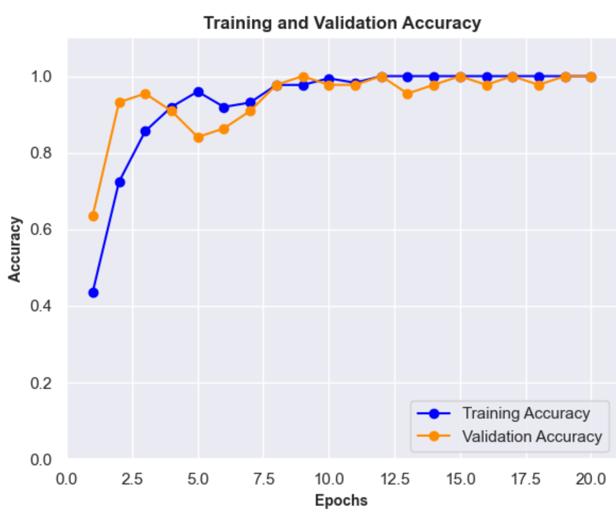
Findings:

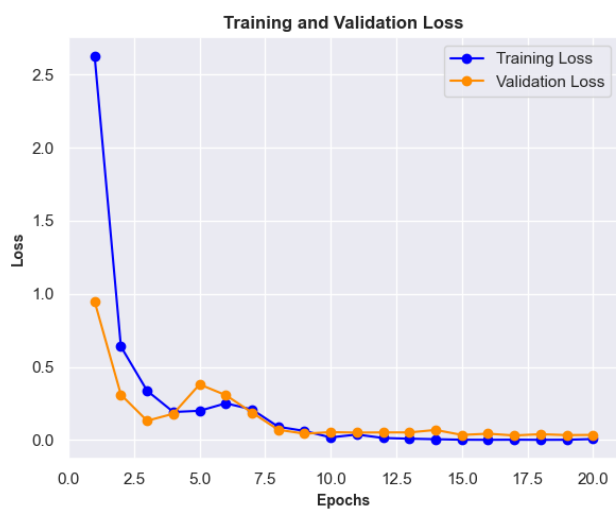
- Geometric augmentation maintained 96.36% accuracy but increased loss to 0.3220.
- Geometric + Blur Augmentation reduced accuracy to 87.27% and Banana recall to 0.611, suggesting that blur may have weakened useful visual details.
- Geometric + Photometric Augmentation maintained 96.36% accuracy with a low loss of 0.0942, indicating stronger prediction confidence among the custom CNN models.

Architecture: The selected model retained the custom 3-block Convolutional Neural Network (CNN) architecture established in the previous hyperparameter tuning experiment. The core modification is the introduction of a robust PyTorch geometric augmentation pipeline applied dynamically to the training data. Since the dataset was small, augmentation settings were kept minimal to preserve key fruit features such as colour, shape, and texture.

- Random rotations up to 15 degrees.
- Random spatial translation (shifting) horizontally and vertically by up to 8% of the image dimensions.
- Random scaling between 92% and 108% of the original image size.
- Random shearing up to 5 degrees to simulate slight perspective distortions.

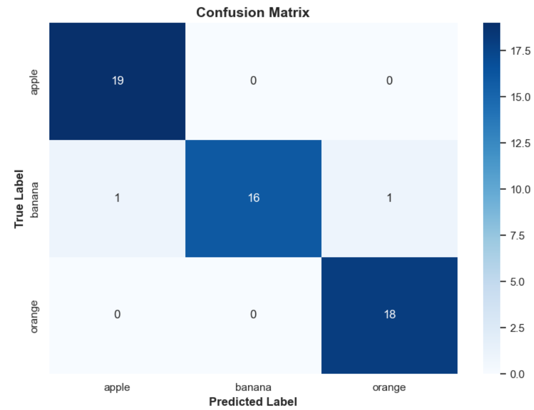
Results:





Class	Precision	Recall	F1-Score	Support
Apple	0.9500	1.0000	0.9740	19
Banana	1.0000	0.8890	0.9410	18
Orange	0.9470	1.0000	0.9730	18

Final Test Loss: 0.3220
Final Test Accuracy: 0.9636



Analysis:

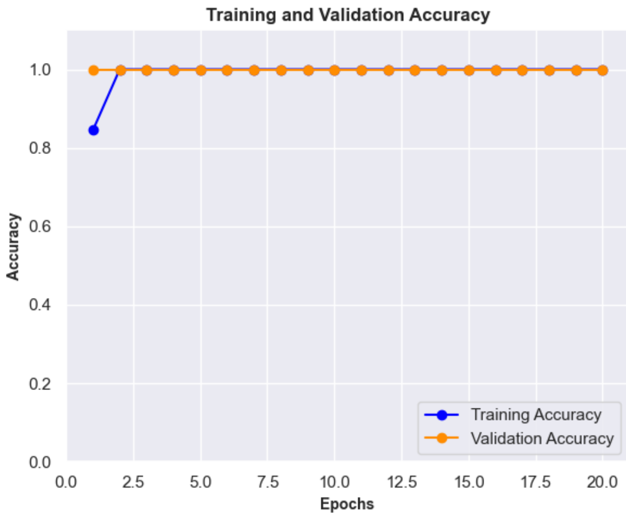
a. **Overall Performance:** The application of geometric augmentation produced a model with highly robust generalization. It achieved a final test accuracy of 0.9636 (96.36%) and a final test loss of 0.3220. This indicates that while the test loss is slightly elevated compared to the unregularized baseline, the augmented model successfully learned structural features, thus reliably matching peak classification accuracy across the dataset.
b. **Mid-Training Instability:** The training process exhibited expected early volatility. Since the model was exposed to random shifting, scaling, and shearing of the images every epoch, validation accuracy briefly dipped around epoch 5, corresponding with minor spikes in validation loss between epochs 5 and 7. However, the model successfully recovered completely by epoch 9 and remained stable through the end of the 20-epoch run.
c. **Signs of Overfitting:** There are no signs of late-stage overfitting. The data augmented-regularization strategy successfully prevented the model from memorizing the specific pixels of the training images. Throughout the full 20 epochs, the training and validation loss curves remained tightly coupled near zero without the upward divergence in validation loss that characterizes overfitting.

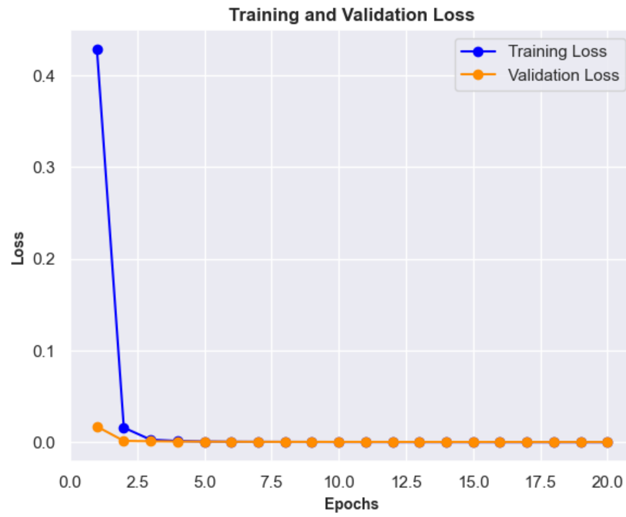
4. Transfer Learning (MobileNetV2) Model

Architecture: This experiment discarded the custom 3-block CNN used in earlier runs. Instead, it leveraged on pre-trained MobileNetV2 model as the foundational feature extractor. The base model weights (pre-trained on ImageNet) were frozen to preserve their learned knowledge. The custom classification head includes:

- A Rescaling layer to scale PyTorch's [0, 1] pixel values to the [-1, 1] range expected by MobileNetV2.
- A GlobalAveragePooling2D layer to compress the spatial dimensions.
- A fully connected Dense layer (128 nodes).
- A 3-node SoftMax output layer.

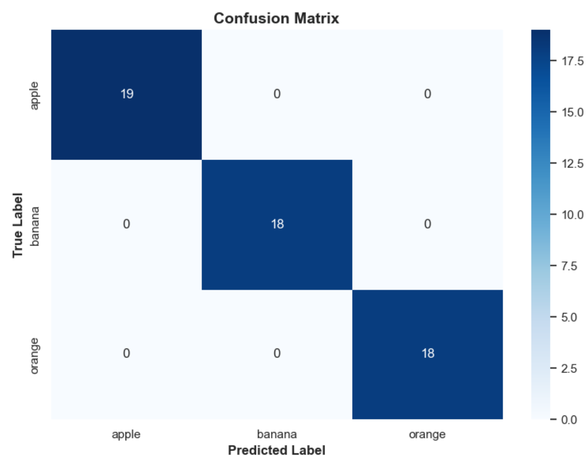
Results:





Class	Precision	Recall	F1-Score	Support
Apple	1.0000	1.0000	1.0000	19
Banana	1.0000	1.0000	1.0000	18
Orange	1.0000	1.0000	1.0000	18

Final Test Loss : 0.0014
Final Test Accuracy : 1.0000



Analysis:

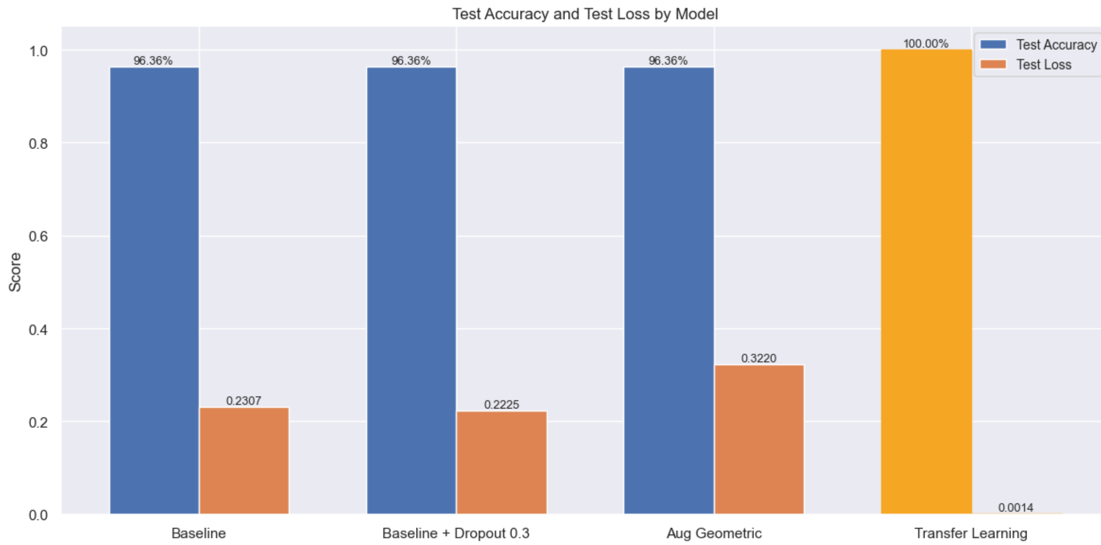
a. **Immediate Convergence:** The transfer learning model demonstrated excellent performance. The visual metrics indicate near-perfect final test accuracy and an exceptionally low-test loss of 0.0014. This demonstrates a massive increase in the model's predictive confidence.

b. **Feature Extraction Efficacy:** From the first epoch, validation accuracy started exceptionally high (approximately 100% at Epoch 1) and remained perfectly stable. Because the MobileNetV2 base already possessed highly optimized feature maps for identifying natural objects, the network did not have to spend time learning basic edges, shapes, or textures; it only needed to map those pre-learned features to your specific fruit classes.

c. **Absence of Overfitting:** The model showed no signs of overfitting over the full 20-epoch run. By Epoch 3, both training and validation loss dropped to near zero and remained tightly coupled and flat through Epoch 20. Because the base model's weights were frozen, the network had few trainable parameters (only the final dense layers). This successfully prevented the network from memorizing the specific training dataset, regularizing the model.

Summary of Findings

1. Effect of Model Optimisation on Predictive Performance



Among our **custom models**, the best model would be Baseline + Dropout 0.3. It shows high stability and robustness even without the use of augmentation. Given that the test accuracy achieved across models were consistent, we selected the Baseline + Dropout 0.3 model as it achieved the same classification performance without augmentation, with lower processing overhead and lower risk of distorting important fruit features.

As for our **transfer learning model**, MobileNetV2 produced the strongest overall result, achieving perfect test accuracy and near-zero test loss, with perfect precision, recall, and F1-scores across all classes. This suggests that the pre-trained feature extractor provided more effective visual features for the current dataset compared to our custom models, especially since MobileNetV2 was pre-trained on a large-scale image dataset.

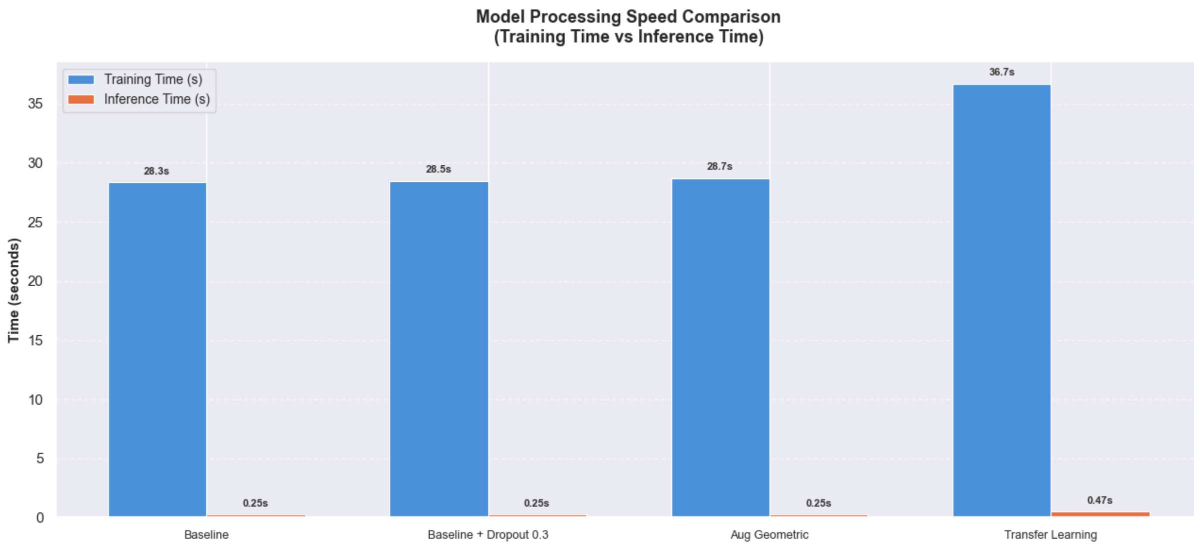
2. Class-Level Behaviour and Misclassification Patterns

The Baseline, Dropout 0.3, Dropout 0.5, Geometric Augmentation, and Photometric Augmentation models all recorded Banana recall of 0.889, while Apple and Orange recall reached 1.000. This indicates a repeated class-specific weakness in Banana classification.

After investigating the misclassified Banana images, we hypothesized that the misclassification was due to visual patterns such as blur, shadow, or colour vibrancy. The augmentation results partially support this hypothesis. Blur augmentation reduced Banana recall to 0.611, while Geometric + Photometric Augmentation improved it to 0.944. This suggests that Banana classification was sensitive to visual detail and augmentation choice.

MobileNetV2 resolved this pattern in the reported test set, achieving 1.000 precision, recall, and F1-score for all classes. However, because the test set contained only 55 images, this should be interpreted as strong evidence for the current dataset, not proof of perfect generalisation.

3. Efficiency and Practical Trade-Offs



Model	Training Time (s)	Inference Time (s)
Baseline	**28.35**	0.2501
Improved Model with Hyperparameter Tuning	28.45	**0.2466**
Geometric Augmentation (w/o Gaussian Blur)	28.70	0.2489
Transfer Learning (MobileNetV2)	36.68	0.4736

The Baseline Model had the fastest training time at 28.35 seconds, while the Dropout 0.3 Model had the fastest inference time at 0.2466 seconds. The Geometric Augmentation Model was similar, with 28.70 seconds training time and 0.2489 seconds inference time. MobileNetV2 required the longest training time at 36.68 seconds and the slowest inference time at 0.4736 seconds. This reflects the higher cost of using a deeper pre-trained feature extractor. However, its inference time remained below one second, so the cost is acceptable when predictive performance is prioritised.

Final Recommendations

Overall, results were satisfactory on the current dataset, achieving above 0.9 test accuracy. MobileNetV2 is recommended as the best pre-trained model, while Dropout 0.3 is recommended as the best-performing custom CNN model.

However, MobileNetV2 may incur high computational cost, as it had the longest training time and slowest inference time. For this project, the trade-off is acceptable because it provided the strongest combined performance. If lightweight deployment is prioritised, Dropout 0.3 is the best alternative, with the best performance and optimization among our custom models.

One main limitation remains that the small dataset size limits how confident the models are in classifying the fruit images. Observations from these experiments should be interpreted with caution; further validation using repeated runs and a larger independent test set is needed before concluding that the models will generalise perfectly.